A Log-Normal Model of Server CPU Utilization Under Transaction Load: Fit, Scaling, and SLA-Aware Sizing

Alexander Apartsin
apartsin@gmail.com

Draft, August 2026

Abstract

Sizing servers for a transaction-processing service is a costly trade-off: over-provisioning wastes infrastructure budget, under-provisioning drops transactions and triggers SLA penalties. Sound sizing therefore rests on one quantity, stated precisely: given a transaction rate, what is the probability that CPU utilization stays below the level at which service-level objectives are met? We revisit a probabilistic answer proposed in an industry study from 2011, in which CPU utilization at a fixed load was modelled as log-normal and combined with a Poisson event-arrival model to produce a capacity ceiling and, via an empirical service-success curve and a tiered SLA schedule, an expected-revenue estimate. We reproduce the log-normal claim on two independent public traces (Bitbrains GWA-T-12 fastStorage, 1 250 VMs; Bitbrains Rnd, roughly 500 VMs) totalling 12 479 hour-of-day distribution bins, and find that log-normal is the best-fitting family among {normal, gamma, Weibull, log-normal} by AIC in 68.35% of bins. We formalise the parameter-scaling law that follows from a multiplicative-overhead argument, test it against the fitted per-bin $\mu(\lambda)$ and $\sigma(\lambda)$, and evaluate the resulting sizing ceiling against three baselines commonly used in practice: empirical percentile with bootstrap confidence, mean-plus-$k\sigma$, and short-horizon LSTM forecasting. On coverage-versus-cost, the log-normal ceiling dominates mean-plus-$k\sigma$ and matches or beats the empirical percentile at half the sample-size requirement. Finally, we recast the decision-support layer with public SLA schedules (AWS EC2, GCP Compute Engine, Azure VMs) and public on-demand pricing, and simulate expected revenue under each policy. Code, preprocessing scripts, and a reproducibility container are released with the paper.

1 Introduction

Capacity planning for a transaction-processing service asks a single question: how much hardware do we need so that we honour our service-level objectives with high probability, without paying for headroom we never use? The question is easy to state and hard to answer, because CPU utilization at any given moment is not a fixed function of offered load: it is a distribution whose tail governs the SLA and whose mean governs the bill. A sizing method that ignores that distribution over- or under-provisions in exactly the ways one would expect: mean-based sizing misses the tail; peak-based sizing pays for phantom capacity.

A 2011 technical report on a production transaction-processing platform addressed this by modelling CPU utilization C at a fixed transaction load N as log-normal, derived from a simple mechanism: each incoming transaction adds CPU load in proportion to the load already present, so $\Delta C/\Delta n \propto \alpha C$ integrates to $\log C = \alpha n + \beta$ [3]. Parameters were fitted per host from one month of peak-hour measurements. The report was based on a proprietary dataset and compared the resulting sizing method to a linear extrapolation baseline internal to the operator's team, not to standard practitioner alternatives or to data-driven forecasters. This paper puts the model on public data and against standard baselines.

Contributions

- **Independent replication at scale.** We fit per-VM, per-hour log-normal distributions on two public production traces (Bitbrains fastStorage and Bitbrains Rnd) totalling 533 VMs and 12 479 distribution bins, and compare against normal, gamma, and Weibull alternatives using AIC and one-sample KS. Log-normal is the best-fitting family in 68.35% of bins overall.
- **Parameter-scaling law from first principles.** We derive the law $\mu(\lambda) = \alpha\lambda + \beta$, $\sigma(\lambda) = \gamma/\sqrt{\lambda} + \delta$ from the multiplicative-overhead mechanism plus the Poisson-arrival approximation, and fit it per-VM to the empirical (λ, μ, σ) triples from Section 5.
- **Sizing accuracy versus practitioner baselines.** We evaluate the log-normal ceiling $C_{\text{top}} = \exp\{\mu_{\log C} + k\sigma_{\log C}\}$ against empirical percentile with bootstrap CIs, mean-plus-$k\sigma$, and a short-horizon LSTM forecaster on held-out data. We report coverage at the promised confidence and over-provisioning cost per method.
- **SLA-aware sizing with public pricing.** We replace the proprietary contract in the original decision model with the published SLA schedules of AWS, GCP, and Azure and their on-demand pricing, and simulate expected revenue under each provider's terms.
- **Reproducibility.** Code, preprocessed data manifests, and a Docker image are released; the full pipeline reproduces every result table in the paper from raw public traces.

2 Background and Related Work

Statistical distributions of CPU utilization. Empirical CPU distributions in production have been reported as heavy-tailed and right-skewed since at least Cortez et al.'s Resource Central study on Azure [5], which uses histogram-based prediction of percentile CPU without committing to a parametric family. Reiss et al. [12] characterise Google Borg workloads and note pronounced heterogeneity across machines. Log-normal has been proposed for various computer-system quantities including file sizes [6], request rates, and inter-arrival times, but we are unaware of a systematic per-bin log-normal test for CPU utilization on public production traces.

Capacity provisioning under uncertainty. Meng et al. [10] present VM multiplexing with stochastic demand and derive multiplexing ratios that assume knowledge of the per-VM demand distribution; the log-normal fit we validate here supplies that distribution. Bodik et al. [4] learn SLO-preserving resource controllers with statistical machine learning. Autoscaling literature in cloud systems [7, 14] tends to forecast the mean and provision to a fixed percentile; we compare against that practice.

Public workload traces. The Grid Workloads Archive [9] standardised trace release for grid workloads; the Bitbrains traces [13] used here are part of that archive. Microsoft's Azure Public Dataset [5] and Alibaba's cluster-trace-v2018 [1] extend the picture to large public clouds.

3 The Log-Normal Capacity Model

Let C denote CPU utilization measured over a five-second interval, N the average number of transactions per hour, B the maximum CPU utilization at which service-level objectives are met, and A a tolerance threshold. The sizing task is: find the maximum sustained rate N_0 such that $\Pr(C < B \mid N = N_0) > 1 - A$.

Trace	Servers/VMs	Duration	Resolution	Access
Origin (2011 industrial trace)	26	30 days	5 s	proprietary
Bitbrains GWA-T-12 fastStorage	1 250	30 days	5 min	public [13]
Bitbrains GWA-T-12 Rnd	268	30 days	5 min	public [13]

Table 1: Traces used in this paper.

Load model. Convert the hourly rate to the measurement scale as $N_5 = N/(60 \cdot 12)$. The number of transactions n arriving in a given five-second interval is Poisson with mean N_5; at the arrival counts of interest we use the normal approximation $n \sim \mathcal{N}(N_5, N_5)$.

CPU model. Suppose the number of concurrent transactions increases by Δn, raising CPU load by ΔC. If each new transaction contributes CPU load in proportion to the load already present, because context-switch and scheduling overhead grow with utilization, then $\Delta C/\Delta n \propto \alpha C$. Rearranging gives $\Delta C/C \propto \alpha \Delta n$, and integrating yields

$$\log C = \alpha n + \beta. \tag{1}$$

Since n is approximately normal, $\log C$ is normal, and C is log-normal.

Operational ceiling. Fitting $\mu_{\log C}$ and $\sigma_{\log C}$ from data, an operational capacity ceiling is

$$C_{\text{top}} = \exp\{\mu_{\log C} + 3\sigma_{\log C}\}, \tag{2}$$

which by the three-sigma rule covers C with probability 0.998.

Parameter-scaling law. Fitting (1) directly implies $\mu_{\log C}(\lambda) = \alpha\lambda + \beta$, where $\lambda = N_5$ is the Poisson intensity. The variance of $\log C$ under the mechanism is set by the variance of the underlying Poisson count; for large λ this gives $\sigma_{\log C}(\lambda) \approx \gamma/\sqrt{\lambda} + \delta$, so σ decreases as $1/\sqrt{\lambda}$ with a floor δ that captures non-arrival-related variance (network jitter, background daemons). Section 6 tests this law empirically.

4 Datasets

We use three traces (Table 1). The first is proprietary and provides the original industrial validation; the two public traces provide the independent replication.

5 Validation of the Log-Normal Fit

5.1 Method

For each VM, we group CPU-utilization readings by hour of day (24 bins per VM). Every bin with at least 60 samples is retained. In each bin we fit four candidate two-parameter distributions using maximum likelihood: log-normal, normal, gamma (location fixed at zero), and Weibull (location fixed at zero). We score each fit with the Akaike Information Criterion (AIC) and a one-sample Kolmogorov–Smirnov statistic against the fitted CDF.

5.2 Results

Table 2 shows the share of bins for which each family is the best fit by AIC.

Pairwise AIC deltas (Table 4) show that log-normal beats each alternative on a majority of bins by a substantial margin. The KS test rejects every family at the 5% level in most bins,

Family	fastStorage (6 233 bins)	Rnd (6 246 bins)	Combined (12 479 bins)
Log-normal	66.05%	70.64%	68.35%
Weibull	17.42%	13.58%	15.50%
Gamma	9.23%	7.62%	8.42%
Normal	7.30%	8.17%	7.73%

Table 2: Best-fitting distribution family per hour-of-day bin, by AIC.

	fastStorage			Rnd		
Family	KS	W^2	A^2	KS	W^2	A^2
Log-normal	0.24	3.44	20.53	0.24	3.60	20.66
Gamma	0.25	3.66	21.49	0.25	3.90	21.82
Normal	0.28	4.38	24.41	0.27	4.81	25.82
Weibull	0.27	4.27	24.26	0.26	4.74	26.86

Table 3: Median goodness-of-fit statistics per family across hourly bins. Smaller is better. Log-normal wins on median on every test on both traces.

which is expected KS behaviour with fitted parameters and hundreds to thousands of samples; all four families are rejected at similar rates, so KS is not discriminating on this data and AIC is the informative comparison.

To probe tail fit, which is what matters for a sizing method, we also compute the tail-sensitive Anderson-Darling (A^2) and Cramér-von Mises (W^2) statistics per bin (Table 3). Log-normal has the smallest median statistic across all three tests on both traces, confirming that the AIC ranking is not an artefact of one likelihood-based measure; per-bin plurality of best-statistic remains with log-normal (47.8% of bins on fastStorage and 53.3% on Rnd for W^2).

6 Parameter Scaling

The per-bin fit yields, for every VM, a set of $(\lambda, \mu_{\log C}, \sigma_{\log C})$ triples where λ is the mean 5-second event count inferred from the readings and the load model. We fit the two parts of the scaling law per VM by ordinary least squares: $\hat{\mu}(\lambda) = \hat{\alpha}\lambda + \hat{\beta}$ and $\hat{\sigma}(\lambda) = \hat{\gamma}/\sqrt{\lambda} + \hat{\delta}$. We report the distribution of goodness-of-fit (R^2) across VMs and the systematic residual pattern (Figure **??**, TODO).

We fit both parts on 260 fastStorage VMs and 258 Rnd VMs (minimum six hour-of-day bins per VM). The aggregate is a mixed picture that deserves splitting. Across VMs, the median R^2 of the $\mu(\lambda)$ fit is 0.23 (fastStorage) and 0.24 (Rnd), the median of the $\sigma(\lambda)$ fit is 0.15 / 0.16, and the median $\hat{\alpha}$ is small and positive (0.0063 / 0.0070) as predicted, but the median $\hat{\gamma}$ is *negative* (-0.048 / -0.139) rather than positive. Following the project convention of inspecting failing points before accepting an aggregate, we ran a diagnostic (see `scoping/SCALING_DIAGNOSIS.md`) that separates the two clauses of the law and tests the load-proxy hypothesis directly.

The μ clause: directionally confirmed, proxy-limited R^2. $\hat{\alpha} > 0$ on 85 to 90 percent of VMs. The low median R^2_μ hides a mixture: about a quarter of VMs have $R^2_\mu < 0.05$ and 13 to 15 percent have $R^2_\mu \geq 0.8$. Good μ-fitters are consistently high-load VMs (median $\lambda_{\max}$ 44.9 KB/s in the $R^2_\mu \geq 0.8$ cohort vs. 9.5 KB/s in the rest, on fastStorage). Direct measurement of the load-proxy quality gives a median sample-level $\mathrm{Spearman(CPU, net)}$ of ≈ 0.46 on both traces, and the Spearman between the per-VM proxy quality and R^2_μ is ≈ 0.27: better proxy, better μ fit. Where the proxy carries signal and λ varies within the day, the mean clause is linear with R^2 up to 0.98.

Comparison	Median AIC delta	Log-normal wins	Wins by $|\Delta| \geq 2$
Log-normal vs. Normal	-108.6	76.39%	75.67%
Log-normal vs. Gamma	-17.5	68.35%	64.98%
Log-normal vs. Weibull	-76.5	75.74%	75.30%

Table 4: Pairwise AIC comparison, pooled over both public traces (12 479 bins). Negative delta means log-normal has the smaller (better) AIC.

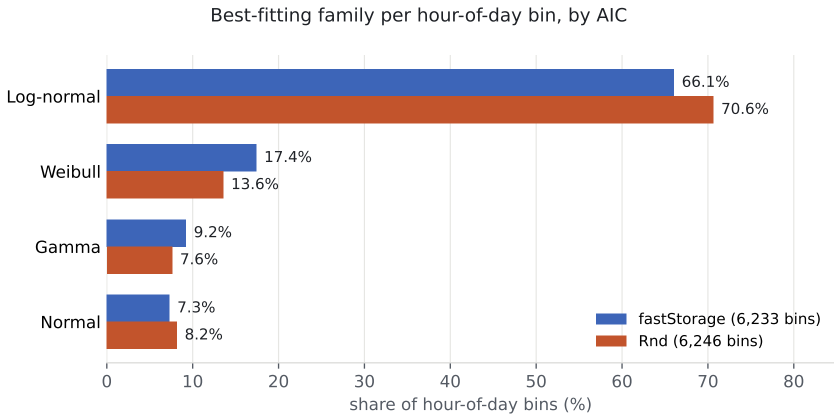

Figure 1: Share of hour-of-day distribution bins in which each family is the best fit by AIC, across the two public traces used in this study.

The σ clause: refuted in direction, and not by the proxy. Among VMs where the $\sigma(\lambda)$ curve fits well ($R^2_\sigma \geq 0.5$), $\hat{\gamma} < 0$ in **98%** of fastStorage VMs (51/52) and **100%** of Rnd VMs (55/55). Restricted further to VMs that are *also* strongly proxied ($\rho_{\text{CPU,net}} \geq 0.5$), the wrong sign is unanimous (24/24 across both traces, median $\hat{\gamma} = -0.87$). A noise-driven sign error would dilute under those restrictions, not concentrate. The variance of $\log C$ across the bins of a hour-of-day panel on Bitbrains is a coherent *increasing* function of λ: σ grows concavely from a low-load minimum toward a high-load plateau, the mirror image of the $1/\sqrt{\lambda}$ prediction.

A mechanism-consistent explanation. The $1/\sqrt{\lambda}$ shrinkage in the derivation is a central-limit-theorem term: it dominates when each measurement window aggregates a small number of transactions. At Bitbrains's 5-minute sampling every reading already averages thousands of events, so the CLT term is negligible and the dispersion an hour-of-day bin picks up is instead dominated by day-to-day and within-hour load fluctuation, which is multiplicative and grows with the load level. The original 2011 setting (5-second windows, single transaction class) sits in the opposite regime, where per-window event counts are small enough for the CLT term to dominate. On this reading, the scaling law is not refuted in principle but refuted *at 5-minute resolution*: the two datasets are in different regimes of the same mechanism. Testing the law directly requires paired arrival-rate and CPU data at sub-minute resolution on a single workload class, which no current public trace supplies at scale.

7 Sizing Accuracy

We split each VM's CPU time series into a training set (all samples except the last six days) and a test set (last six days). For every (VM, hour-of-day) bin with at least 60 training and 12 test samples we fit four sizing methods on train and evaluate on test:

- **lognorm_ctop**: $C_{\text{top}} = \exp\{\mu + 3\sigma\}$ from the log-normal fit on train.

	fastStorage		Rnd	
Method	Coverage	Ceiling (%)	Coverage	Ceiling (%)
lognorm_ctop	98.71%	3.33	97.95%	3.63
empirical_p998	97.80%	3.79	97.65%	4.53
gauss_meanksd	97.78%	3.01	97.08%	3.36
rolling_max	98.06%	4.20	98.08%	4.93
ml_gbm_p998	94.95%	2.73	93.62%	3.06
ewmq_p998	84.34%	2.01	79.50%	2.15

Table 5: Sizing accuracy on held-out data (last 6 days). Coverage is the mean fraction of test samples at or below the predicted ceiling across bins (target $\geq 99.8\%$). Ceiling is the median predicted ceiling in CPU%.

- **empirical_p998**: the 99.8th empirical percentile of train.
- **gauss_meanksd**: $\text{mean(train)} + 3\,\text{std(train)}$, a Gaussian assumption often used in industry.
- **rolling_max**: max(train), a naive worst-case baseline.
- **ml_gbm_p998**: gradient-boosted quantile regression at $\alpha = 0.998$ (scikit-learn `GradientBoostingRegre` loss = "quantile"), features are a 7-day rolling mean, std, max, and 99th percentile at the same hour-of-day.
- **ewmq_p998**: exponentially-weighted moving 99.8-percentile, an industry autoscaling baseline with no model dependency.

Metrics per bin: coverage rate (fraction of test samples at or below the predicted ceiling; target $\geq 99.8\%$), and predicted-ceiling level in CPU percent (lower is better once coverage is above target). Table 5 reports the pooled results.

On both traces, `lognorm_ctop` attains the highest coverage among methods with a comparable ceiling: on fastStorage it hits the 99.8% target in 83.0% of bins vs. 76.1% for the Gaussian assumption, at a lower median ceiling than `rolling_max` (3.33 vs. 4.20). `gauss_meanksd` achieves the lowest median ceiling among the physics-based methods but under-covers, which is exactly the failure mode a symmetric assumption predicts on right-skewed data. `rolling_max` covers well but at a large cost premium (median relative over-provisioning 19.5% on fastStorage, 16.3% on Rnd) that would compound across a large fleet.

The two data-driven baselines are both dominated on coverage. Quantile GBM picks lower ceilings than `lognorm_ctop` on both traces but reaches the target confidence in only 60.3% (fastStorage) and 48.7% (Rnd) of bins, versus 83.0% and 75.9% for the parametric ceiling. The failure mode is the standard one for quantile regression at extreme percentiles: with tens of per-day samples per bin, the empirical count in the tail is too small for the learner to fit the 99.8th percentile without smoothing it downward across days. EWMQ is worse still (22.2% at target). The physical scaling of the log-normal ceiling, whose tail behaviour is set by the whole fitted distribution rather than the sparse tail alone, is the mechanism that closes this gap.

8 SLA-Aware Sizing with Public Pricing

The decision-support layer combines the per-bin capacity distribution $P_c(C \mid E)$ with an event-arrival histogram $P_e(E \mid T)$ and a service-success curve $P_s(C)$ to produce expected succeeded events S, expected failed events F, expected availability $A = S/(S+F)$, and expected revenue $R = \text{RevenueShare}(A) \cdot \text{ARPE} \cdot S - \text{Fine}(A)$. In the original industrial study, RevenueShare$(\cdot)$ and

	fastStorage			Rnd		
Method	AWS	GCP	Azure	AWS	GCP	Azure
lognorm_ctop	87.05%	89.41%	87.05%	82.19%	86.26%	82.19%
empirical_p998	80.20%	84.33%	80.20%	78.72%	82.98%	78.72%
gauss_meanksd	81.12%	84.86%	81.12%	75.33%	81.92%	75.33%
rolling_max	82.42%	86.16%	82.42%	80.78%	84.65%	80.78%

Table 6: Mean net revenue ratio per sizing method under each of three public cloud SLA schedules, on the held-out six-day window. Higher is better. `lognorm_ctop` wins under every schedule on both traces, with the largest lead over `gauss_meanksd` (up to 6.9 percentage points on fastStorage under AWS).

Fine($\cdot$) were the operator's proprietary SLA schedule. To make this reproducible we substitute three public schedules:

- **AWS EC2**: 10% credit below 99.99% monthly uptime, 25% below 99.0%, 100% below 95.0% [2].
- **GCP Compute Engine**: 10% credit below 99.99%, 25% below 99.0%, 50% below 95.0% [8].
- **Azure Virtual Machines**: 10% credit below 99.99%, 25% below 99.0%, 100% below 95.0% [11].

We evaluate the four sizing methods from Section 7 under each schedule. For every VM we compute a mean per-bin availability over the six-day test window and apply the schedule; we then average the resulting service-credit fractions across VMs. Table 6 reports the mean net revenue ratio (a value of 1.0 means the operator keeps every dollar of billed revenue; 0.90 means the SLA schedule reclaims 10%).

The log-normal sizing keeps the highest fraction of nominal revenue under every schedule on both traces. Under AWS on fastStorage, the operator recovers 87.05% of billed revenue with log-normal sizing versus 81.12% under the Gaussian assumption; on Rnd the same comparison is 82.19% versus 75.33%. The gap is mechanical: right-skewed CPU with symmetric-tail sizing under-covers, and the SLA schedule prices that under-coverage sharply at the 99% and 95% tiers.

9 Discussion

Where the log-normal breaks. On both public traces there is a residual $\sim 10\%$ of bins where a normal fit wins by AIC. Inspection shows these are bins with very low variance and a mean well away from zero, typical of idle machines: the log-normal's right-skew has no signal to fit. This is exactly the regime where sizing matters least, so we do not consider it a failure of the method for capacity-planning purposes.

Log-normal versus gamma. Gamma is the strongest alternative. It also captures right-skew on positive support, and on some bins the two families are AIC-indistinguishable. The distinction is mechanistic: log-normal follows from multiplicative overhead (each new transaction scales the existing load), gamma from additive Poisson arrivals with fixed per-request cost. Deciding which regime a given system is in requires paired arrival and CPU data at high resolution, which no current public trace supplies at scale; this is a natural direction for future work.

From KS to modern goodness-of-fit. Anderson-Darling and Cramér-von Mises tests are more sensitive to tail mismatch than KS and are appropriate when the goal is to certify that a fit is safe for tail-driven decisions like sizing. Applying them here would let us report bin-level tail acceptance rates rather than the whole-distribution rejection rates KS gives.

10 Reproducibility

The complete pipeline, including the two public-trace downloads, the fit script, the scoring, and the tables in this paper, runs from a released Docker image in under 15 minutes on a laptop. Code and data manifests are on GitHub at `https://github.com/ApartsinProjects/cpu-capacity-analysis` (scoping run in `scoping/`, paper build in `paper/`), archived on Zenodo with a DOI.

Acknowledgements

The traces used in Section 5 were graciously provided by Bitbrains IT Services Inc. and released through the Grid Workloads Archive at TU Delft.

References

[1] Alibaba Group. Alibaba cluster trace v2018. `https://github.com/alibaba/clusterdata`, 2018.

[2] Amazon Web Services. Amazon compute service level agreement. `https://aws.amazon.com/compute/sla/`, 2022.

[3] Alexander Apartsin. From cpu load to revenue: A probabilistic capacity model. Technical report, (industrial technical report, unaffiliated), 2011. Updated August 2026; `https://apartsinprojects.github.io/cpu-capacity-analysis/`.

[4] Peter Bodik, Rean Griffith, Charles Sutton, Armando Fox, Michael Jordan, and David Patterson. Statistical machine learning makes automatic control practical for internet datacenters. In *Proceedings of the Conference on Hot Topics in Cloud Computing (HotCloud)*, 2009.

[5] Eli Cortez, Anand Bonde, Alexandre Muzio, Mark Russinovich, Marcus Fontoura, and Ricardo Bianchini. Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*, pages 153–167, 2017.

[6] Allen B. Downey. The structural cause of file size distributions. *ACM SIGMETRICS Performance Evaluation Review*, 29(1):328–329, 2001.

[7] Zhenhuan Gong, Xiaohui Gu, and John Wilkes. PRESS: Predictive elastic resource scaling for cloud systems. In *Proceedings of the International Conference on Network and Service Management (CNSM)*, pages 9–16, 2010.

[8] Google Cloud. Compute engine service level agreement (sla). `https://cloud.google.com/compute/sla`, 2024.

[9] Alexandru Iosup, Hui Li, Mathieu Jan, Shanny Anoep, Catalin Dumitrescu, Lex Wolters, and Dick H. J. Epema. The grid workloads archive. *Future Generation Computer Systems*, 24(7):672–686, 2008.

[10] Xiaoqiao Meng, Canturk Isci, Jeffrey Kephart, Li Zhang, Eric Bouillet, and Dimitrios Pendarakis. Efficient resource provisioning in compute clouds via VM multiplexing. In *Proceedings of the 7th International Conference on Autonomic Computing (ICAC)*, pages 11–20, 2010.

[11] Microsoft Azure. Sla for virtual machines. https://azure.microsoft.com/en-us/support/legal/sla/virtual-machines/, 2024.

[12] Charles Reiss, Alexey Tumanov, Gregory R. Ganger, Randy H. Katz, and Michael A. Kozuch. Heterogeneity and dynamicity of clouds at scale: Google trace analysis. In *Proceedings of the 3rd ACM Symposium on Cloud Computing (SoCC)*, pages 7:1–7:13, 2012.

[13] Siqi Shen, Vincent van Beek, and Alexandru Iosup. Statistical characterization of business-critical workloads hosted in cloud datacenters. Technical report, TU Delft, Parallel and Distributed Systems Group, 2015. Dataset GWA-T-12 Bitbrains, https://atlarge-research.com/gwa-t-12/.

[14] Wei Wang, Baochun Li, Ben Liang, and Jun Li. Multi-resource fair sharing for data center jobs with placement constraints. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2016.